Lecture 33 — SVD and capstone launch

Fri Dec 4

Last updated: October 29, 2026.

Textbook: Boyd & Vandenberghe, *VMLS* (PDF)

Strang / MIT (optional): Reading: Strang/MIT SVD slides (PDF)
Video: 18.06 Lec 29 — singular value decomposition · Short SVD overview (Strang) · MIT 18.06 playlist (Strang)

Reading: selected slides from the Strang/MIT SVD slides (not all 17 pages), capstone launch, Lachniet Ch. 5 (SVD) and Ch. 7 (image compression). Related objectives: Lectures. Schedule.

Bring paper and a laptop. Warm-up, fill-in notes, board examples, and **Your turn**. Today is **SVD + Lab 6 (Fit and Compress) + project launch** after Quiz 11. See the schedule.

Learning objectives

By the end of class, you should be able to:

1. Interpret $A = U\Sigma V^\mathsf{T}$ conceptually.
2. Construct a rank-k approximation computationally.
3. Compare singular-value decay for an image/frame and a frame-difference matrix.
4. Encode three frames either independently or as F_1, D_2, D_3.
5. Compare reconstruction error and storage for the two approaches.
6. Formulate a capstone question involving accuracy, storage, runtime, and resource use.

(Objectives 3–6 are launched today and carried in the capstone packet.)

Warm-up (after quiz)

A grayscale image (or one video frame) is an $m \times n$ matrix of pixel intensities.

(a) Roughly how many numbers store the full matrix?

(b) If most "information" lived in a few patterns, why might a factored low-rank form save storage?

Fill in

SVD (conceptual). Every real $m \times n$ matrix factors as

$$A = U\Sigma V^\mathsf{T},$$

where the columns of U and V are orthonormal (left / right **singular vectors**) and Σ is rectangular "diagonal" with nonnegative **singular values**

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$$

on its main diagonal. Think: A = rotate/reflect $\rightarrow$ stretch by σ_i $\rightarrow$ rotate/reflect.

Factorizations in this course. Lab 3 introduced a first useful factorization: $A = LU$ (elimination saved as a product — built to **solve** $Ax = b$). Today's **SVD** is a second: built from **orthonormal** directions and singular values — built to **approximate**, **compress**, and **compute least squares stably**.

Orthonormal payoff (Lecture 14). "Orthonormal" still means $q_i^\mathsf{T} q_j = 0$ for $i \neq j$ and $\|q_i\| = 1$. The SVD **hands you** orthonormal bases (columns of U and of V). That is why those bases were worth defining earlier: coordinates via inner products, and this factorization, both use them.

Two jobs for SVD. (1) **Stable LS:** the LS solution is still $\hat{b} = \text{proj}_{C(A)}(b)$ (Lectures 38–39); SVD is a reliable way to **compute** $\hat{x}$ (software's `lstsq` often uses this family). (2) **Compression:** keep only the largest σ_i for a rank-k picture (capstone).

Callback — eigenvectors (Lectures 35–37, Lab 5). Eigenvectors name directions that A only **rescales** ($Av = \lambda v$); the largest $|\lambda|$ wins the long run. Singular vectors / singular values name directions that A **stretches** the most ($\sigma_1 \geq \sigma_2 \geq \cdots$). Different jobs, same "**keep the dominant directions**" instinct: Lab 5's $\lambda = 1$ mode vs today's keep-large-σ_i compression. (For square symmetric cases the two stories are closely related; we do not need that theorem to use either tool.)

Rank-k approximation. Keep the largest k singular values (and matching singular-vector columns); set the rest to zero. The truncated product

$$A_k = U_k \Sigma_k V_k^\mathsf{T}$$

is the best rank-k approximation in the usual 2-norm / Frobenius sense (details in the slides / capstone). Same idea as "keep the important orthonormal directions."

Software sketch (lab / capstone):

Task	Octave	Python
SVD	`[U,S,V] = svd(A)`	`U, S, Vt = np.linalg.svd(A, full_matrices=False)`
Rank-k rebuild	use first k columns / σ's	`U[:,:k] @ np.diag(S[:k]) @ Vt[:k,:]`
Least squares	`A\b`	`np.linalg.lstsq(A,b,rcond=None)[0]`

Capstone launch (one question). For a short clip of frames: when is it better to compress each frame alone versus encode F_1 and differences $D_2 = F_2 - F_1$, $D_3 = F_3 - F_2$? Trade off **accuracy**, **storage**, **runtime**, and **resource use** — including the cost of computing the SVD itself. Full prompts live in the capstone packet; studio is Mon Dec 7.

Worked in class (short)

W1. Read Σ. Suppose a 4×3 matrix has singular values $\sigma = (10, 2, 0.1)$. What is rank(A) (numerically "full" vs tiny)? For a rank-1 approximation, which σ do you keep?

W2. Orthonormal check (Lecture 14 vocabulary). If u_1, u_2 are two columns of U, what should $u_1^\top u_2$ and $\|u_1\|$ be?